School of Future Tech

ITM Skills University

Case Study 86 – Report

Subject: Java Programming

on

Smart Home Appliance Control Simulator

Java Swing GUI, OOP, Collections, Threads & Exception Handling

Desktop Application Development in Java

by

Subrata Bhaktipad Panda — 150096725110

B.Tech Computer Science Engineering and AI | Semester III

Cohort: Larry Page

Academic Year 2025–2029

Report

Java | Swing | Collections Framework | Runnable & Thread | Custom Exceptions

Index

1	Abstract	3
2	Introduction	3
3	Objectives	3
4	Problem Statement	3
5	System Components and Requirements	4
6	Tools & Technologies	4
7	System Architecture and Folder Structure	5
8	Requirement-to-Module Mapping	6
9	Class Design (Model Layer)	7
10	Service Layer: ApplianceManager	7

11	Collections, Searching and Sorting	8
12	Exception Handling	9
13	Simulation using Runnable and Thread	9
14	Swing GUI Design	10
15	Testing and Verification Plan	12
16	Results and Conclusion	12

1. Abstract

The Smart Home Appliance Control Simulator is a Java-based desktop application that models a simple smart home control system. It lets a user register household appliances such as lights, fans and air conditioners, map each appliance to a room, switch devices on and off, track power consumption, view a history of power events, search and sort the appliance list, and generate usage reports.

The project demonstrates core Java programming concepts: abstraction through an abstract **Appliance** class, an interface (**Controllable**), inheritance, the Collections Framework (ArrayList, LinkedList, HashMap, TreeMap), custom exception handling, a Swing graphical user interface, and thread-based simulation using **Runnable** and **Thread**. All data is held in memory and the application runs entirely on a single machine, so it needs no hardware or database.

2. Introduction

A smart home connects everyday appliances to a central controller so that they can be monitored and switched remotely. Real systems depend on IoT hardware and cloud services, but the software ideas behind them (modelling devices, tracking state, validating limits, logging events) can be practised in a pure Java simulator.

This report presents the design and implementation of such a simulator. Each requirement of the case study is traced to a module, a class and a test, so that the report shows what was built, why it was built that way and how it was checked.

The design principles used throughout are: model real-world entities as classes, keep common behaviour in one abstract base class, separate data, logic and display into different packages, validate input before changing state, and keep the GUI responsive while a simulation thread runs in the background.

3. Objectives

- **Appliance and room information:** store device ID, name, type, room, status and power rating, and map appliances to rooms.
- **Remote control:** turn any appliance on or off from the GUI.
- **Power tracking:** calculate the power drawn by each device and the total load, and enforce the 5000 W limit.
- **Real-time simulation:** change the state of several devices automatically using a separate thread. •

Search and sort: search by name, room, type and status; sort by power, name and status. • **GUI:**

provide a Swing control panel with tabs for appliances, rooms, history, reports and simulation. •

Reports and history: generate a usage summary and keep a log of on/off events.

4. Problem Statement

A smart home platform needs a simulator to control and monitor household appliances such as lights, fans and air conditioners remotely. The system should track each appliance's on/off status, room mapping, power consumption and state changes, and it should simulate real-time state changes across multiple devices.

Expected outcomes:

• A class model with an abstract base class, an interface and three appliance subclasses

. • A service class that performs add, find, control, search, sort, validation, logging and reporting • A Swing GUI that exposes these operations through tabs and buttons.

• A background simulation that changes device states safely while the GUI is open.

• A test plan showing that every objective works as expected.

Scope. The project is a single-user desktop simulator. It does not connect to real IoT hardware, cloud storage or an external database. Sample data is created inside the application and every change stays in memory until the window is closed.

5. System Components and Requirements

The table summarises the main components of the system.

Appliance model	Abstract class, inheritance	Appliance is the base class; Light, Fan and AirConditioner extend it with their own properties.
Remote control contract	Interface	Controllable declares the turn-on and turn-off operations that every appliance must provide.
Room mapping	HashMap	Rooms are stored by room ID and each room holds the IDs of its appliances.
Power tracking	Arithmetic + validation	Current load is the sum of the ratings of all appliances that are ON; it must not exceed 5000 W.
Event history	LinkedList of PowerLog	Every on/off change is stored as a PowerLog entry with device, state and time.
Search and sort	ArrayList, TreeMap, Comparator	Search by name, room, type and status; sort by power, name and status.
Simulation	Runnable and Thread	DeviceSimulator repeatedly toggles devices on a background thread.
User interface	Java Swing	A tabbed window with tables, buttons, dialogs and a status bar showing the current load.
Error handling	Custom exceptions	InvalidDeviceException and PowerLimitExceededException report bad input and overload.

6. Tools & Technologies

Language	Java (JDK 8 or later)	Core application logic and object-oriented design.
GUI toolkit	Java Swing (javax.swing, java.awt)	Windows, tabbed panes, tables, buttons and dialogs.

Collections	java.util: ArrayList, LinkedList, HashMap, TreeMap	Storing appliances, rooms, history and sorted views.
Concurrency	Runnable, Thread, SwingUtilities	Background simulation and safe GUI updates.
Build and run	javac, java, run.sh	Compile all sources into the out folder and start app.Main.
Documentation	Markdown / Word / PDF	Project report and README.

7. System Architecture and Folder Structure

The application uses a layered structure. The **model** package holds data classes, the **service** package holds business logic, the **ui** package holds the Swing interface, the **simulation** package holds the background thread, and the **exception** package holds custom error types. **app.Main** starts the program.

app	Main	Entry point; creates the manager, loads sample data, opens the GUI.
model	Appliance, Controllable, Light, Fan, AirConditioner, Room, PowerLog	Data and device behaviour.
service	ApplianceManager	CRUD, room mapping, control, search, sort, validation, logging, reports.
simulation	DeviceSimulator	Implements Runnable; simulates state changes on a thread.
ui	SmartHomeGUI	Swing window with tabs, tables, buttons and the status area.
exception	InvalidDeviceException, PowerLimitExceededException	Checked exceptions for invalid devices and power overload.

7.1 Folder structure

```
SmartHomeSimulator/
 |-- README.md
 |-- run.sh
 |-- Documentation/
 |-- screenshots/
 |-- src/
 | |-- app/
 | | `-- Main.java
 | |-- exception/
 | | |-- InvalidDeviceException.java
 | | `-- PowerLimitExceededException.java
 | |-- model/
 | | |-- AirConditioner.java
 | | |-- Appliance.java
 | | |-- Controllable.java
 | | |-- Fan.java
 | | |-- Light.java
 | | |-- PowerLog.java
 | | `-- Room.java
 | |-- service/
 | | `-- ApplianceManager.java
```

```
| |-- simulation/
| | `-- DeviceSimulator.java
| `-- ui/
| `-- SmartHomeGUI.java
`-- out/
```

7.2 Architecture diagram

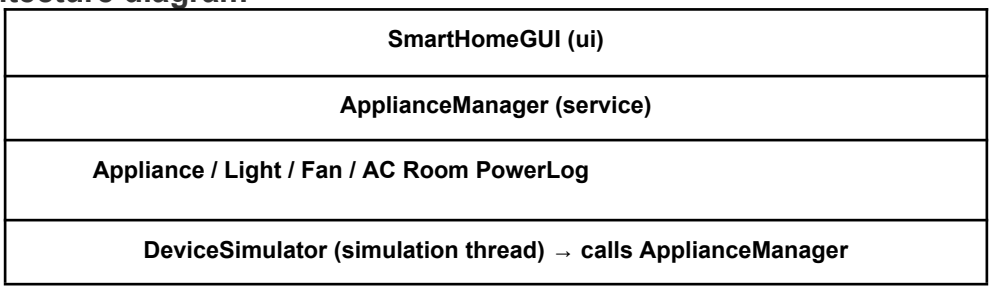


Figure 7.1 — Layered architecture of the simulator

8. Requirement-to-Module Mapping

Each objective of the case study is handled by a specific module. This table links requirement, module, class and Java concept.

1	Register lights, fans, air conditioners	Appliance Registration	Appliance, Light, Fan, AirConditioner	Abstract class, inheritance, constructors
2	Map appliances to rooms	Room Mapping	Room, ApplianceManager	HashMap, ArrayList
3	Turn devices on and off	Remote Control	Controllable, Appliance	Interface, polymorphism
4	Track power and enforce 5000 W limit	Power Tracking	ApplianceManager	Validation, custom exception
5	Record on/off events	State Logging	PowerLog	LinkedList
6	Search and sort records	Search and Sort	ApplianceManager	Comparator, TreeMap, ArrayList
7	Show usage summary	Usage Reports	ApplianceManager	StringBuilder, iteration
8	Simulate real-time changes	Simulation	DeviceSimulator	Runnable, Thread
9	Provide a control GUI	User Interface	SmartHomeGUI	Swing, event listeners

9. Class Design (Model Layer)

The **Appliance** abstract class stores the data common to every device. **Light**, **Fan** and **AirConditioner** extend it and add properties of their own. The **Controllable** interface is implemented by Appliance so that any device can be controlled through the same two methods.

Controllable (interface)	turnOn(), turnOff()	Contract for remote control.
Appliance (abstract)	deviceId, deviceName, roomId, isOn, powerRating; getters; abstract getType()	Common data and behaviour of all appliances.

Light	brightness	Lighting device with a brightness level.
Fan	speed	Fan with a speed setting.
AirConditioner	temperature	Air conditioner with a target temperature; highest power rating.
Room	roomId, roomName, list of appliance IDs	Groups appliances by location.
PowerLog	deviceId, state, timestamp	One entry of the event history.

9.1 Illustrative code: base class and interface

```
public interface Controllable {
 void turnOn();
 void turnOff();
}

public abstract class Appliance implements Controllable {
 private final String deviceId;
 private final String deviceName;
 private final String roomId;
 private final int powerRating; // watts
 private boolean on;

 public Appliance(String id, String name, String roomId, int watts) {
 this.deviceId = id; this.deviceName = name;
 this.roomId = roomId; this.powerRating = watts;
 }
 public void turnOn() { on = true; }
 public void turnOff() { on = false; }
 public boolean isOn() { return on; }
 public int getCurrentPower() { return on ? powerRating : 0; }
 public abstract String getType();
}

public class Fan extends Appliance {
 private int speed;
 public Fan(String id, String name, String roomId, int watts, int speed) {
super(id, name, roomId, watts);
 this.speed = speed;
 }
 public String getType() { return "Fan"; }
}
```

Note: the listing is a simplified outline of the model classes; replace it with the exact code from the project if required.

10. Service Layer: ApplianceManager

ApplianceManager is the only class that changes application data. The GUI and the simulator both call it, so validation, power limits and logging are applied in one place.

Add / find / remove appliance	Checks for a duplicate or empty ID, then stores the device in an ArrayList and a HashMap keyed by device ID. Throws InvalidDeviceException when the ID is unknown or invalid.
Room mapping	Adds the device ID to the Room object in the room HashMap; rooms are created on demand.
Control (turn on / off)	Looks up the device, calculates the load if it were switched on, throws PowerLimitExceededException if the total would exceed 5000 W, otherwise changes the state and adds a PowerLog entry.
Power calculation	Adds getCurrentPower() of every appliance to give the total current load in watts.
Search	Filters the list by name, room, type or status (case-insensitive).

Sort	Returns a sorted copy by power, name or status using a Comparator.
Report	Builds a text summary: number of devices, devices ON, total load and the most recent events.

10.1 Illustrative code: power validation

```
public void turnOnDevice(String id) throws InvalidDeviceException,
 PowerLimitExceededException {
 Appliance a = devices.get(id);
 if (a == null) throw new InvalidDeviceException("No device with ID " + id);
if (!a.isOn() && getTotalPower() + a.getPowerRating() > POWER_LIMIT)
 throw new PowerLimitExceededException(
 "Turning on " + a.getDeviceName() + " exceeds " + POWER_LIMIT + " W");
a.turnOn();
 history.add(new PowerLog(id, "ON", System.currentTimeMillis()));
 }
```

11. Collections, Searching and Sorting

Each collection class was chosen for the operation it performs best.

ArrayList	Ordered list of appliances shown in the GUI table	Fast indexed access and easy iteration for tables and sorting.
LinkedList	Power event history	Events are appended continuously and the latest ones are read from the end.
HashMap	Device ID → Appliance, Room ID → Room	Constant-time lookup by ID.
TreeMap	Appliances ordered by power consumption	Keeps keys sorted automatically.

11.1 Search and sort options

Search	Name, room, type, status	Loop over the list and keep matches (contains, ignoring case).
Sort by power	Rating or current power (high to low)	Comparator on power value, or TreeMap.
Sort by name	Alphabetical	Comparator using compareToIgnoreCase.
Sort by status	ON devices first, then OFF	Comparator on the boolean state.

```
// sort by power, highest first
list.sort((a, b) -> Integer.compare(b.getPowerRating(), a.getPowerRating()));

// sort by name
list.sort(Comparator.comparing(Appliance::getDeviceName, String.CASE_INSENSITIVE_ORDER));
```

12. Exception Handling

Two custom exceptions make errors specific and easy to display. The GUI catches them and shows a dialog instead of letting the program crash.

InvalidDeviceException	The ID is empty, duplicated or not found, or numeric input is invalid.	GUI shows an error dialog; no data changes.
PowerLimitExceededException	Switching a device on would push the total load above 5000 W.	GUI shows a warning; device stays OFF.
NumberFormatException	Power rating field is not a number.	Caught in the Add dialog and converted to a user message.

```
try {
 manager.turnOnDevice(selectedId);
 refreshTable();
} catch (InvalidDeviceException | PowerLimitExceededException ex) {
 JOptionPane.showMessageDialog(frame, ex.getMessage(),
 "Cannot complete action", JOptionPane.WARNING_MESSAGE);
}
```

13. Simulation using Runnable and Thread

DeviceSimulator implements **Runnable**. Its run() method loops while the simulation is active, picks a device, toggles its state through ApplianceManager, and then sleeps for a short interval. Running this loop in its own Thread keeps the Swing event thread free, so the window stays responsive.

- **Start / stop:** a volatile boolean flag lets the GUI stop the loop cleanly.
- **Safe updates:** after each change the simulator asks the GUI to refresh through SwingUtilities.invokeLater.
- **Shared data:** manager methods that change state are synchronized so the GUI and the simulator cannot corrupt the same list.
- **Limit respected:** if a toggle would exceed 5000 W the exception is caught and the simulator moves to the next device.

```
public class DeviceSimulator implements Runnable {
 private volatile boolean running = true;
 public void stop() { running = false; }

 public void run() {
 while (running) {
 try {
 manager.toggleRandomDevice();
 ui.refresh();
 Thread.sleep(1500);
 } catch (InterruptedException e) {
 Thread.currentThread().interrupt();
 return;
 } catch (PowerLimitExceededException | InvalidDeviceException e) {  // skip
this device and continue
 }
 }
```

14. Swing GUI Design

SmartHomeGUI is a single JFrame. A status area at the top shows the current power usage, and a JTabbedPane holds the five working screens.

Appliances	Table of all devices with ID, name, type, room, status and power	Turn ON, turn OFF, Add appliance, Search, Sort
Rooms	Rooms and the appliances mapped to each	View room contents

Power History	List of on/off events from the LinkedList	Refresh history
Reports	Text area with the usage summary	Generate report
Simulation	Start and stop buttons for DeviceSimulator	Start / stop simulation

14.1 Screenshots

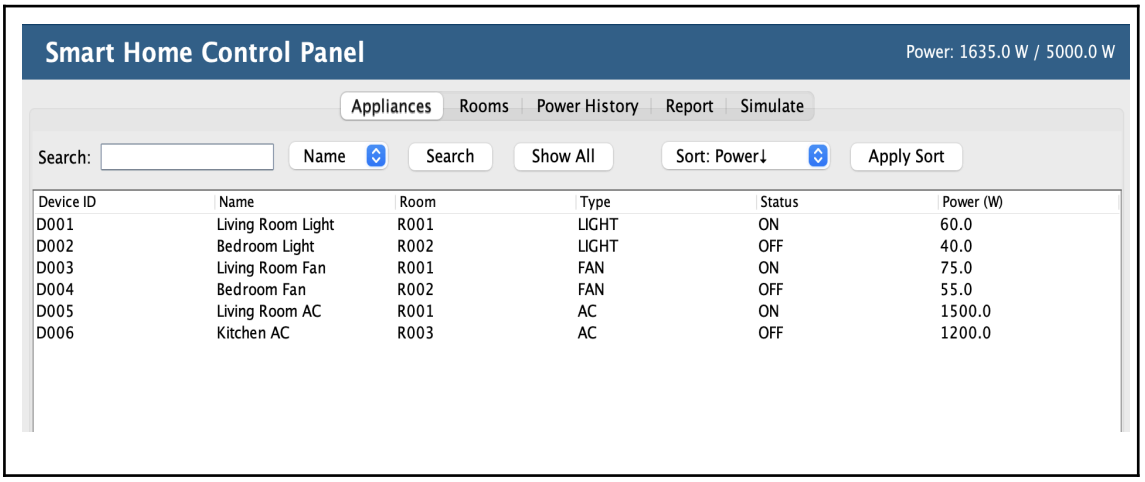


Figure 14.1 — Main appliance dashboard

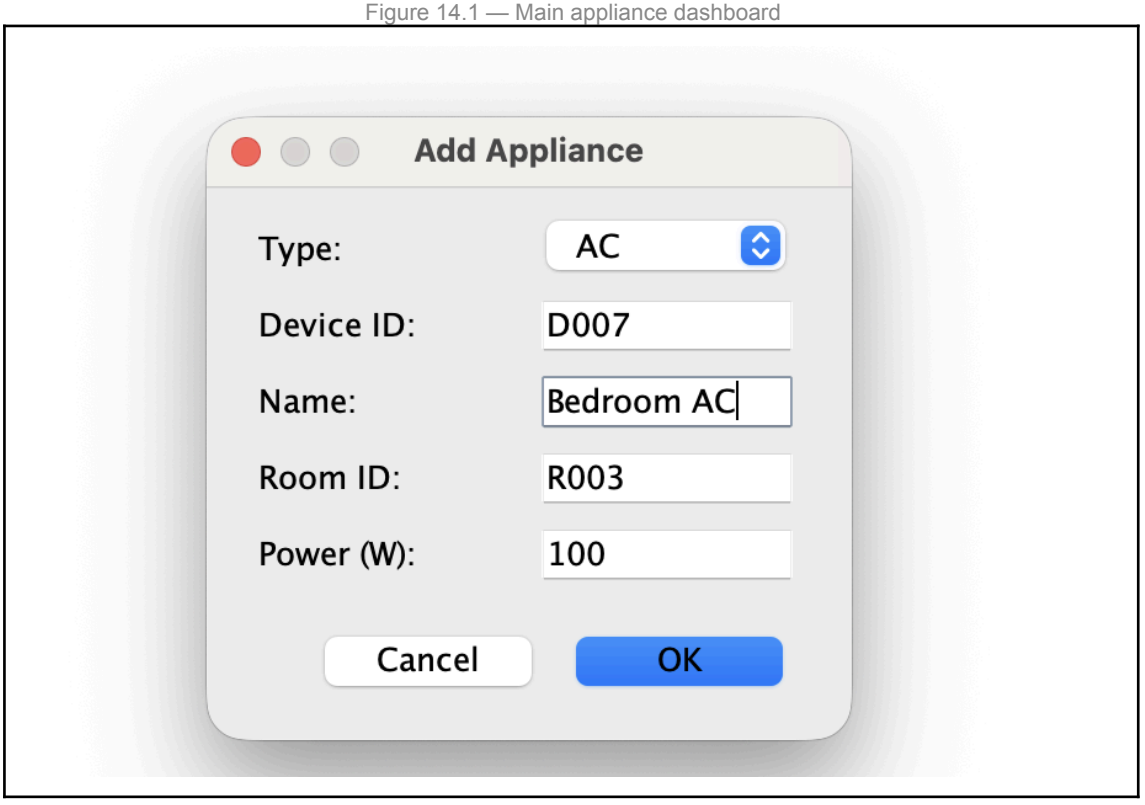


Figure 14.2 — Add appliance dialog

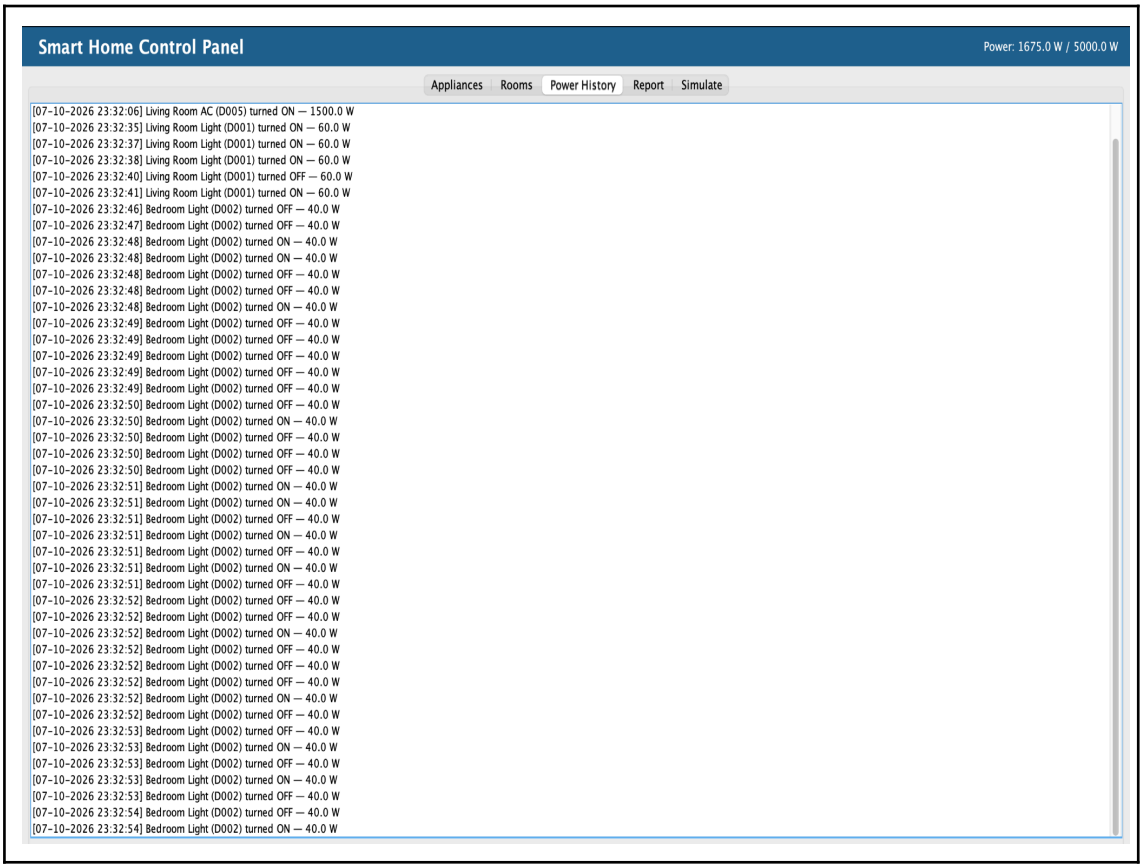
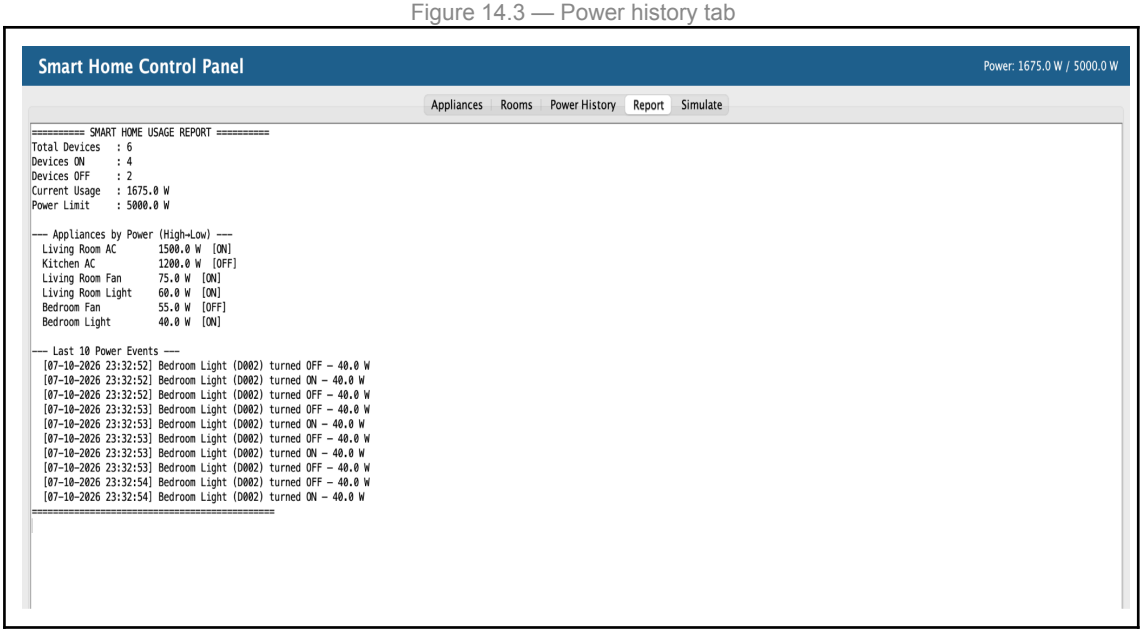


Figure 14.3 — Power history tab



15. Testing and Verification Plan

The project is built with the following commands from the project root.

```
bash run.sh

# manual build and run
mkdir -p out
javac -d out $(find src -name "*.java")
java -cp out app.Main
```

Each objective is verified with a test action and an expected result.

1	Compilation	Run javac on all source files	No errors; class files appear in out/
2	Appliance and room data	Start the application	Sample appliances and rooms are listed
3	Remote control	Select a device and click Turn ON, then Turn OFF	Status changes and the table refreshes
4	Power tracking	Turn devices ON one by one	Top status area shows the updated total in watts
5	Power limit	Turn on devices until the total would pass 5000 W	Warning dialog appears; device stays OFF
6	Add appliance	Open Add dialog and enter valid details	New device appears in the table and in its room
7	Invalid input	Add a device with an empty or duplicate ID, or a non-numeric power	Error message; nothing is added
8	Search	Search for a room name, type or status	Only matching appliances are shown
9	Sort	Sort by power, name and status	Rows reorder as selected
10	History	Open Power History after changes	Each on/off event is listed in order
11	Report	Click Generate report	Summary shows device count, devices ON, total load and recent events
12	Simulation	Start the simulation	States change automatically while the GUI stays responsive; Stop ends it

The results above are the expected behaviour of the application; record the observed results and attach the matching screenshots when the tests are run.

16. Results and Conclusion

16.1 Findings

Appliance and room data maintained	Appliance hierarchy, Room class, HashMap and ArrayList	Sections 9, 10
Remote on/off control	Controllable interface and ApplianceManager control methods	Sections 9, 10, 14
Power consumption tracked and limited	Total load calculation and PowerLimitExceededException	Sections 10, 12
Search and sort	Filtering and Comparator-based sorting, TreeMap	Section 11
Real-time simulation	DeviceSimulator on a separate thread	Section 13
GUI, reports and history	Swing tabs, LinkedList of PowerLog, report generator	Section 14

16.2 Limitations

- Data is stored only in memory and is lost when the application closes.
- The simulator does not talk to real devices, a network or a cloud service.
- It supports a single user with no login or access control.
- Only three appliance types are available.

16.3 Future enhancements

- Add persistent storage with files or a database.
- Add charts for power consumption trends.
- Add user login and role-based access.
- Add appliance category filters.
- Export reports to PDF or CSV directly from the GUI.

16.4 Conclusion

The Smart Home Appliance Control Simulator meets the objectives of the case study. It provides a working Swing interface for managing appliances and rooms, controls devices remotely, tracks and limits power use, keeps an event history, supports searching and sorting, generates usage reports and simulates real-time changes on a background thread. The project shows how abstraction, interfaces, inheritance, collections, exception handling, threads and Swing work together in a complete Java application.